INFORME DEL PROYECTO

Parcial 2 Corte

Sistema de gestión de proyectos de grado

Institución	Universitaria Tecnológica de Santander (UTS)
Estudiantes	Alejandro Melgarejo Hernandez
	Jhoan Alejandro Rodriguez
Tecnologías	JSP, JSTL, SQL y MariaDB/MySQL

Índice

1. Presentación del proyecto **3**
1.1. Qué resuelve la aplicación . . . 3
1.2. Ficha técnica . . . 3
1.3. Alcance de la revisión . . . 3

2. Herramientas, estructura y arquitectura **3**
2.1. Inventario de archivos y responsabilidades . . . 3
2.2. Arquitectura de ejecución . . . 4
2.3. Servidor y bibliotecas . . . 5
2.4. Recorrido típico de una operación POST . . . 5

3. Modelo entidad-relación y esquema de la base de datos **5**
3.1. Paso 1. Crear la base y cargar los catálogos . . . 5
3.2. Paso 2. Modelo entidad-relación del PDF de requerimientos . . . 6
3.2.1. Lectura del modelo conceptual . . . 8
3.3. Paso 3. Modelo físico creado por los scripts SQL . . . 8
3.4. Paso 4. Relaciones físicas y cardinalidades principales . . . 9
3.5. Paso 5. Diferencias entre el modelo conceptual y las tablas reales . . . 9
3.6. Paso 6. Restricciones que afectan el funcionamiento . . . 10

4. Recorrido por el código: login, sesión y administración **11**
4.1. Paso 7. Inicio de sesión en `login.jsp` . . . 11
4.2. Paso 8. Cerrar la sesión en `logout.jsp` . . . 12
4.3. Paso 9. Administrar usuarios en `admin-usuarios.jsp` . . . 12

5. Recorrido por el código: módulos del estudiante **13**
5.1. Paso 10. Banco de ideas y panel en `index.jsp` . . . 13
5.2. Paso 11. Solicitar idea del banco . . . 14
5.2.1. Reserva y política de tiempo . . . 15
5.3. Paso 12. Corregir y cancelar una solicitud . . . 15
5.4. Paso 13. Registrar idea estudiantil en `crear-idea.jsp` . . . 16

6. Recorrido por el código: coordinación y revisión académica **16**
6.1. Paso 14. Coordinación revisa comprobantes y solicitudes . . . 17
6.2. Paso 15. Coordinación revisa ideas . . . 18

6.3. Paso 16. Docentes proponen ideas institucionales 19
6.4. Paso 17. Revisión del anteproyecto 20
6.5. Paso 18. Revisión del proyecto final 20

7. Entregas estudiantiles y seguimiento 21
7.1. Paso 19. Enviar anteproyecto y proyecto final 21
7.2. Paso 20. Revisar y reenviar una entrega aplazada 21
7.3. Fases visibles en `project-phase-tracker.jsp` 22

8. Código del proyecto: datos y operaciones principales 22
8.1. Paso 21. Cargar las ideas semilla desde SQL 22
8.2. Reserva y solicitud de una idea en `solicitudes-ideas.jsp` 23
8.3. Envío de anteproyecto desde `anteproyecto.jsp` 23
8.4. Revisión de anteproyecto y escritura del historial 24

9. Puesta en marcha paso a paso 25
9.1. Paso 22. Instalar y cargar la base 25
9.2. Paso 23. Configurar Tomcat y JNDI 25
9.3. Paso 24. Guion de prueba por roles 25

10. Errores frecuentes y diagnóstico 27

11. Requisitos frente a la implementación 28

12. Conclusiones 28

Fuentes e inventario consultados 29

1. Presentación del proyecto

1.1 Qué resuelve la aplicación

Parcial 2 Corte es un sistema web de gestión del proceso de proyectos de grado. Sus páginas JSP cubren la consulta del banco de ideas, la propuesta de temas por docentes y estudiantes, la selección mediante solicitud, la revisión del comprobante de pago, la asignación de dirección y evaluación, y la entrega escalonada del anteproyecto y del proyecto final. Los estados y las revisiones se guardan en la base de datos para que cada panel muestre el avance del proceso.

Este informe describe el comportamiento que se encuentra en el código entregado y distingue esa implementación del modelo conceptual incluido en el PDF de requerimientos. El diagrama de requisitos se reproduce, pero no se presenta como si fuera una exportación de las tablas actuales: el modelo conceptual y el SQL físico tienen diferencias que se explican más adelante.

1.2 Ficha técnica

Elemento	Detalle observado
Interfaz y control	Páginas JSP con JSTL Core, Functions y SQL; no se observan Servlets propios
Acceso a datos	Recurso JNDI `jdbc/ProjectsDB` y operaciones JSTL SQL parametrizadas
Motor de datos	Esquema `proyectos_grado_uts`, declarado compatible con MariaDB 10.4 / MySQL 8
Contenedor esperado	Tomcat y API `javax.servlet`, según el descriptor de despliegue
Modelo físico	22 tablas y la vista SQL `ideas_aprobadas`
Vistas web	Diez páginas JSP principales y un fragmento JSP de seguimiento
Recursos de interfaz	CSS propio en `assets/css/styles.css`; JSTL 1.2 y conector MySQL incluidos
Datos semilla	Cuatro roles, seis áreas, tres programas y diez ideas institucionales de ejemplo

1.3 Alcance de la revisión

Se inspeccionaron las páginas JSP, los scripts SQL, el descriptor de despliegue, la configuración JNDI, las bibliotecas y los requerimientos. No se ejecutó el proyecto en Tomcat contra una base de datos activa. Por ello, la explicación del comportamiento está basada en el código presente; la disponibilidad de cuentas, el resultado de la carga semilla y el funcionamiento en producción requieren pruebas en el entorno de despliegue.

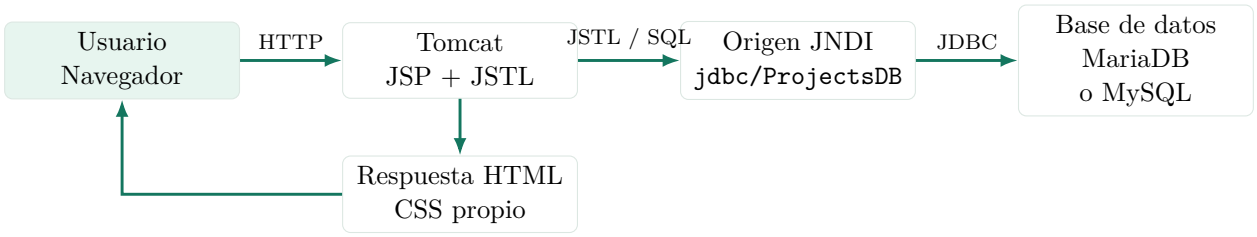
2. Herramientas, estructura y arquitectura

2.1 Inventario de archivos y responsabilidades

Archivo o carpeta	Responsabilidad observada
`login.jsp`	Formulario de inicio de sesión, validación de credenciales y redirección por rol.
`logout.jsp`	Elimina las variables de autenticación y el token CSRF de la sesión; vuelve al login.
`index.jsp`	Inicio del estudiante, búsqueda del banco aprobado, propuestas propias y plazos de entrega.
`crear-idea.jsp`	Registro de una propuesta estudiantil junto con la solicitud y el comprobante asociado.
`solicitudes-ideas.jsp`	Solicitud de una idea, reserva, corrección del comprobante y cancelación.
`coordinador-ideas.jsp`	Revisión administrativa/académica de ideas y solicitudes; asignación de evaluador.
`docente.jsp`	Propuesta institucional y revisión de anteproyectos y proyectos finales asignados.
`anteproyecto.jsp`	Envío/corrección estudiantil de anteproyecto y proyecto final.
`admin-usuarios.jsp`	Alta, modificación y desactivación de cuentas, roles y perfiles.
`usuarios-publicos.jsp`	Directorio de información pública de usuarios.
`database/01_create_schema.sql`	Creación de base, tablas, función de hash y vista de ideas aprobadas.
`database/02_insert_initial_data.sql`	Inserción de catálogos y diez ideas semilla con prerequisitos de docentes.
`database/legacy/`	Migraciones de instalaciones previas; se conservan aparte del esquema nuevo.
`META-INF/context.xml`	Configuración del origen JNDI JDBC que utiliza el código JSP.
`WEB-INF/web.xml`	Referencia al recurso, timeout de sesión y opciones de cookies/seguimiento.
`WEB-INF/lib/`	Librería JSTL 1.2 y conectores MySQL y PostgreSQL; el contexto de la aplicación elige MySQL.
`WEB-INF/project-phase-tracker.jsp`	Fragmento de seis etapas insertado en paneles del flujo académico.
`assets/css/styles.css`	Estilos de login, navegación, paneles, formularios, avisos y pantallas adaptables.

2.2 Arquitectura de ejecución

El proyecto se organiza alrededor de páginas JSP: la petición llega a una JSP; las etiquetas JSTL comprueban sesión, rol y campos; las etiquetas SQL ejecutan las lecturas o escrituras con el origen JNDI; y la página devuelve HTML con el CSS de la aplicación.



2.3 Servidor y bibliotecas

`WEB-INF/web.xml` usa la configuración de Servlet 3.1 y los tipos `javax.servlet`. Esto corresponde al estilo de Tomcat 9 y no debe tratarse como compatible automáticamente con Tomcat 10 u 11, donde la API pasó a `jakarta.servlet`. El archivo `META-INF/context.xml` configura el controlador `com.mysql.cj.jdbc.Driver`. En `WEB-INF/lib/` se encuentra el conector MySQL, junto con JSTL 1.2.

El descriptor fija una sesión de 20 minutos, limita el seguimiento a cookies y marca la cookie como HTTP-only. La referencia `jdbc/ProjectsDB` se define en el descriptor y se configura en el contexto de Tomcat; las sentencias SQL de las JSP usan ese mismo nombre.

2.4 Recorrido típico de una operación POST

1. El navegador envía los campos y un parámetro `action` a la página JSP que contiene el formulario.
2. La página comprueba sesión, rol, token CSRF y validez de campos.
3. Una o más operaciones `<sql:query>` y `<sql:update>` consultan/modifican filas. Las acciones relacionadas se agrupan con `<sql:transaction>`.
4. La página usa filas afectadas o resultado de consulta para distinguir éxito, reglas de negocio incumplidas y error de base de datos.
5. En varios flujos se guarda un aviso temporal en la sesión y se hace una redirección, de modo que recargar la página no vuelva a enviar el POST.

El código no contiene un JSPF de conexión JDBC basado en `DriverManager`, ni un `AccessFilter` propio. El acceso a la base se centraliza en el recurso JNDI del contenedor; los controles de sesión y rol se realizan dentro de cada JSP.

3. Modelo entidad-relación y esquema de la base de datos

3.1 Paso 1. Crear la base y cargar los catálogos

El primer archivo de datos se debe ejecutar antes que las páginas: `database/01_create_schema.sql` crea la base `proyectos_grado_uts` con codificación `utf8mb4` y define sus tablas. Luego `database/02_insert_initial_data.sql` inicializa roles, áreas y programas. Entre los programas están DSI (Desarrollo de Sistemas Informáticos), MRA (Manejo de Recursos Ambientales) y OMEM (Operación y Mantenimiento Electromecánico).

```
CREATE DATABASE IF NOT EXISTS proyectos_grado_uts
  CHARACTER SET utf8mb4
```

```
  COLLATE utf8mb4_unicode_ci;
USE proyectos_grado_uts;

INSERT IGNORE INTO roles (nombre, descripcion) VALUES
  ('administrador', ...),
  ('coordinador', ...),
  ('docente', ...),
  ('estudiante', ...);
```

3.2 Paso 2. Modelo entidad-relación del PDF de requerimientos

La figura siguiente corresponde al modelo entidad-relación que acompaña el documento de requerimientos del proyecto. Se reproduce la figura original para conservar las entidades, relaciones y cardinalidades tal como fueron suministradas.

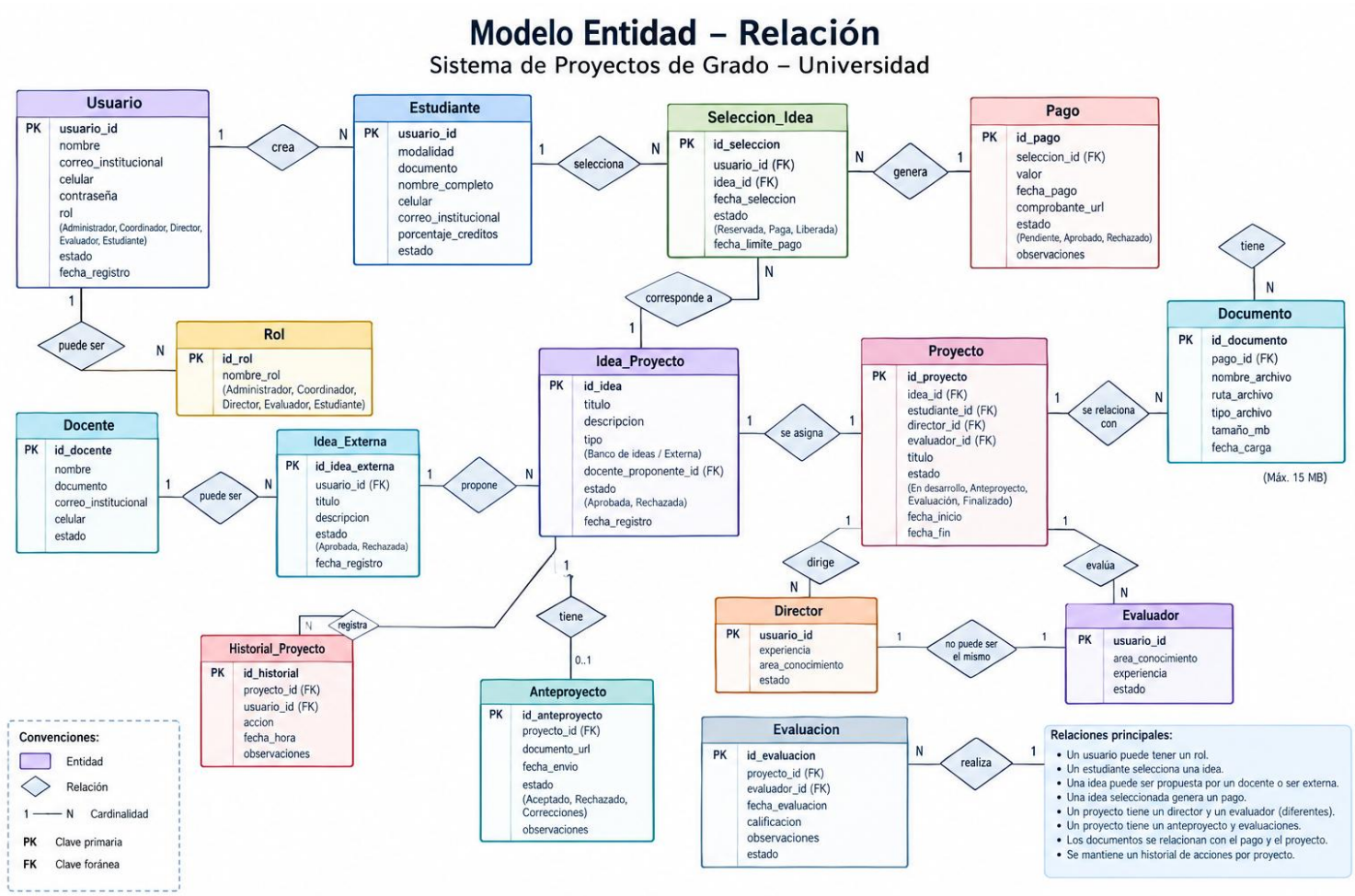


Figura 1. Modelo entidad-relación conceptual de requerimientos. Fuente: `REQUERIMIENTOS.pdf`.

3.2.1. Lectura del modelo conceptual

El diagrama representa las entidades Usuario, Rol, Estudiante, Docente, Idea Externa, Idea Proyecto, Selección de Idea, Pago, Documento, Proyecto, Director, Evaluador, Anteproyecto, Evaluación e Historial de Proyecto. Sus relaciones principales vinculan la selección con estudiante e idea, el pago con la selección, el documento con el pago, la idea con el proyecto, los roles con usuarios y las revisiones con sus responsables.

Atención: este modelo es una referencia conceptual, no la estructura exacta creada por el script SQL de la aplicación. La implementación física separa o consolida varias de esas responsabilidades. A continuación se documentan las tablas reales que consultan las páginas.

3.3 Paso 3. Modelo físico creado por los scripts SQL

El esquema contiene 22 tablas InnoDB y una vista de lectura. Este inventario recoge las tablas definidas por `01_create_schema.sql`:

Grupo y tablas físicas	Qué datos representan en el sistema
Identidad y catálogos	
`roles`, `usuario_roles`, `usuarios`	Nombre de rol; asociación N:M; correo, hash y sal de contraseña, estado y fechas de acceso/registro.
`programas_academicos`, `areas_tematicas`	Catálogos académicos con código, descripción y bandera de activo.
Perfiles	
`estudiantes`	Extiende usuario con código estudiantil, programa, semestre y fecha de ingreso.
`docentes`	Extiende usuario con departamento y perfil profesional.
Ideas y solicitudes	
`ideas`	Código, título, problema, descripción, tipo, estado, programa, área, proponente, selección y fechas.
`revisiones_ideas`	Historial de decisiones y observaciones de coordinación sobre la idea.
`direcciones_ideas`, `evaluadores_ideas`	Docente director y evaluador asignados, quién los asignó y cuándo.
`solicitudes_ideas`	Estudiante líder/compañero, mensaje, URL del comprobante, marca de pago, estado, respuesta y plazo de corrección.
Entregas y revisiones académicas	
`anteproyectos`, `revisiones_anteproyectos`	Anteproyecto y sus revisiones por director/evaluador con decisiones, observaciones y fechas.
`proyectos_finales`, `revisiones_proyectos_finales`	Entrega final, su estado de revisión y el historial de decisiones por etapa.
Seguimiento general del proyecto	

Grupo y tablas físicas	Qué datos representan (continuación)
`proyectos`, `integrantes_proyecto`, `asesores_proyecto`	Proyecto académico, estudiantes participantes y docentes asesores.
`entregables`, `notificaciones`, `historial_proyectos`	Archivos/enlaces de entregables, avisos por usuario y transiciones históricas de estado.

Además del conjunto de tablas, el archivo crea la vista `ideas_aprobadas`. Esta vista selecciona ideas cuyo estado es `aprobada` y presenta junto a ellas el programa, el área y el usuario proponente. `index.jsp` consulta esta vista para montar el banco disponible.

3.4 Paso 4. Relaciones físicas y cardinalidades principales

Relación física	Card.	Implementación
Usuario – rol	N:M	Tabla puente `usuario_roles`; clave primaria `(usuario_id, rol_id)`.
Usuario – estudiante/docente	1:0..1 por subtipo	La clave del perfil es también FK a `usuarios.id`.
Programa – estudiante	1:N	FK `estudiantes.programa_id`.
Programa/área – idea	1:N	FKs `ideas.programa_id` y `ideas.area_id`.
Usuario – idea proponente	1:N	FK `ideas.proponente_id`.
Estudiante – idea seleccionada	1:0..1 selección vigente	`ideas.seleccionada_por` referencia `estudiantes.usuario_id` y es UNIQUE.
Idea – revisión / dirección / evaluador	1:N / 1:0..1 / 1:0..1	`revisiones_ideas` guarda eventos; las dos asignaciones usan `idea_id` como clave.
Idea – solicitudes	1:N	FK en `solicitudes_ideas.idea_id`; la pareja idea/estudiante no se duplica.
Estudiante – solicitud	1:N	El líder es obligatorio; el integrante es opcional y también referencia a estudiante.
Idea-estudiante – anteproyecto	1:0..1 por pareja	Restricción única `(idea_id, estudiante_id)`.
Anteproyecto – proyecto final	1:0..1	`proyectos_finales.anteproyecto_id` es UNIQUE.
Entrega – revisiones	1:N	Tablas independientes de revisión para anteproyecto y proyecto final.
Proyecto – tablas relacionadas	1:N por relación	Tablas relacionadas por `proyecto_id`.

3.5 Paso 5. Diferencias entre el modelo conceptual y las tablas reales

Entidad del ER de requisitos	Cómo se representa en el SQL actual	Qué significa al leer el código
`Seleccion_Idea`	Reserva en las columnas de `ideas`; solicitud en `solicitudes_ideas`.	No se crea tabla física `Seleccion_Idea`.
`Pago`	`comprobante_pago_url` y `pago_verificado` viven en la solicitud.	No hay entidad de pago, monto ni integración financiera independiente.
`Documento`	URL en solicitudes, anteproyectos, proyectos finales o entregables.	Los formularios aceptan enlaces compartidos; no se observa carga binaria de archivos en los JSP.
`Idea_Externa`	`ideas.tipo='estudiantil'` distingue una propuesta estudiantil.	Es una modalidad de la tabla `ideas`, no una tabla de idea externa.
`Director/Evaluador`	Asignaciones en `direcciones_ideas` y `evaluadores_ideas`, ambas referidas al perfil docente.	El seed inicial define cuatro roles. Director y evaluador se expresan como funciones asignadas a docentes.
`Evaluacion`	Las decisiones se guardan en tablas de revisiones separadas por fase.	No hay tabla genérica `Evaluacion`.
`Historial_Proyecto`	`historial_proyectos`.	El esquema la declara; eso no garantiza que los paneles JSP actuales expongan todas sus operaciones.
`Rol` del diagrama	`roles` más la relación `usuario_roles`.	La relación física admite N:M, aunque la figura conceptual muestra un rol por usuario.

3.6 Paso 6. Restricciones que afectan el funcionamiento

- El correo, el código estudiantil, el código de idea y los códigos/nombres de los catálogos se protegen mediante índices `UNIQUE`.
- La clave primaria compuesta de `usuario_roles` evita asignar dos veces la misma pareja de usuario y rol.
- La unicidad `(idea_id, estudiante_id)` de solicitudes evita repetir la misma solicitud. Los formularios pueden actualizar una solicitud retirada o vencida en lugar de insertar otra.
- La pareja de anteproyecto y el anteproyecto de proyecto final se protegen también con claves únicas para prevenir entregas duplicadas.
- Las claves foráneas enlazan perfiles, proponentes, revisor, estudiante, director, evaluador, programa, área, proyecto y entregas.
- Los campos `ENUM` acotan los estados válidos del workflow: por ejemplo idea, solicitud, anteproyecto y proyecto final manejan catálogos de estados diferentes.

4. Recorrido por el código: login, sesión y administración

4.1 Paso 7. Inicio de sesión en `login.jsp`

1. Si hay `userId` en la sesión, la página consulta de nuevo el usuario activo y su rol. Las cuentas vigentes se envían a su panel; las sesiones inválidas se limpian.
2. El formulario POST presenta el token CSRF guardado en sesión. Si falta o no coincide, no se evalúan las credenciales.
3. La consulta busca coincidencia de correo sin distinguir mayúsculas, requiere cuenta activa y sal presente, y compara el hash almacenado contra la función del esquema.
4. Una rama `<c:when>` por rol guarda `userId`, `userName` y `userRole`, regenera el token y redirige.
5. El estudiante llega al banco; el docente, a `docente.jsp`; coordinación, a `coordinador-ideas.jsp`; administración, a `admin-usuarios.jsp`.

```
WHERE LOWER(u.correo)=LOWER(?) AND u.estado='activo'
  AND u.salt_contrasena IS NOT NULL
  AND u.hash_contrasena=pg_password_digest(?, u.salt_contrasena)
...
<sql:param value="${param.email}" />
<sql:param value="${param.password}" />
```

La función `pg_password_digest` está definida dentro del script SQL y realiza iteraciones SHA-256 sobre contraseña y sal. La explicación aquí documenta el mecanismo del proyecto; no certifica su resistencia criptográfica.

Una vez consultado el usuario, la JSP establece la sesión y elige el panel de acuerdo con el rol devuelto:

```
<sql:query var="newCsrf" dataSource="jdbc/ProjectsDB">
  SELECT LOWER(SHA2(CONCAT(UUID(),RAND(),UUID(),NOW(6)),256))
    AS token
</sql:query>
<c:set var="csrfToken" value="${newCsrf.rows[0].token}"
       scope="session" />
<c:when test="${not empty loginResult.rows
    and loginResult.rows[0].rol eq 'docente'}">
  <c:set var="userId" value="${loginResult.rows[0].id}"
         scope="session" />
  <c:set var="userName"
         value="${loginResult.rows[0].nombre_completo}"
         scope="session" />
  <c:set var="userRole" value="${loginResult.rows[0].rol}"
         scope="session" />
  <c:set var="csrfToken" value="${newCsrf.rows[0].token}"
         scope="session" />
  <c:redirect url="/docente.jsp" />
</c:when>
```

Las ramas equivalentes envían cada rol a su panel: administración a `admin-usuarios.jsp`; coordinación

a `coordinador-ideas.jsp`; y estudiante a `index.jsp`.

4.2 Paso 8. Cerrar la sesión en `logout.jsp`

`logout.jsp` elimina las cuatro variables de sesión `userId`, `userName`, `userRole` y `csrfToken`, y redirige al login. El tiempo límite de inactividad de la sesión está definido en 20 minutos en `web.xml`.

```
<c:remove var="userId" scope="session" />
<c:remove var="userName" scope="session" />
<c:remove var="userRole" scope="session" />
<c:remove var="csrfToken" scope="session" />
<c:redirect url="/login.jsp" />
```

4.3 Paso 9. Administrar usuarios en `admin-usuarios.jsp`

Esta página comprueba que el usuario tenga una sesión activa y rol administrador. Los formularios usan el campo `action` y se enrutan a una de tres ramas:

- **`create`:** valida nombre, formato simple del correo, rol de la lista de cuatro roles, contraseña entre 8 y 200 caracteres y, para estudiantes, código académico y programa activo. En una transacción crea cuenta, sal/hash, rol y perfil correspondiente.
- **`update`:** valida los nuevos datos y el estado. Si no se entrega contraseña, conserva el hash; si se cambia, deriva un nuevo hash y sal. Actualiza asociación de rol y el perfil del estudiante/docente.
- **`delete`:** aplica baja lógica cambiando estado a inactivo, no borra la fila. Impide desactivar la sesión propia y protege la última cuenta administradora activa.

La contraseña y la sal se derivan en SQL; el correo se normaliza a minúsculas. Si una restricción única rechaza el cambio, la página comunica que correo o código pueden estar registrados.

Acción `create`: alta de cuenta y perfil. El ejemplo muestra la inserción de la cuenta con una sal aleatoria y el hash producido por la función del esquema. Las siguientes consultas de la misma transacción asignan el rol y, según corresponda, crean el perfil de estudiante o docente.

```
<sql:transaction dataSource="jdbc/ProjectsDB">
  <sql:update var="createdCount">
    INSERT INTO usuarios
      (correo,nombre_completo,hash_contrasena,salt_contrasena,estado)
    SELECT LOWER(?),?,
      proyectos_grado_uts.pg_password_digest(?,salts.salt),
      salts.salt,'activo'
    FROM (
      SELECT LOWER(SUBSTRING(
        SHA2(CONCAT(UUID(),UUID(),RAND(),RAND()),256),1,32
      )) AS salt
    ) salts
    <sql:param value="${fn:trim(param.email)}" />
    <sql:param value="${fn:trim(param.name)}" />
    <sql:param value="${param.password}" />
  </sql:update>
```

```
  <sql:query var="createdUserId">
    SELECT LAST_INSERT_ID() AS id
  </sql:query>
  <sql:update>
    INSERT INTO usuario_roles (usuario_id,rol_id)
    SELECT ?,id FROM roles WHERE nombre=?
    <sql:param value="${createdUserId.rows[0].id}" />
    <sql:param value="${param.role}" />
  </sql:update>
</sql:transaction>
```

Acción `update`: conservar o renovar contraseña. Si el campo llega vacío, la consulta modifica los datos básicos y conserva el hash; cuando se recibe una contraseña nueva, genera otra sal y vuelve a calcular el hash.

```
<c:when test="${empty param.password}">
  UPDATE usuarios
  SET correo=LOWER(?),nombre_completo=?,estado=?
  WHERE id=?
</c:when>
<c:otherwise>
  UPDATE usuarios u
  JOIN (SELECT LOWER(SUBSTRING(
    SHA2(CONCAT(UUID(),UUID(),RAND(),RAND()),256),1,32
  )) AS salt) salts
  SET u.correo=LOWER(?),u.nombre_completo=?,u.estado=?,
      u.salt_contrasena=salts.salt,
      u.hash_contrasena=
        proyectos_grado_uts.pg_password_digest(?,salts.salt)
  WHERE u.id=?
</c:otherwise>
```

Acción `delete`: baja lógica. No se borra la fila ni su historial: tras comprobar que no sea la propia cuenta ni el último administrador activo, el código cambia el estado.

```
UPDATE usuarios
SET estado='inactivo'
WHERE id=? AND estado<>'inactivo'
```

5. Recorrido por el código: módulos del estudiante

5.1 Paso 10. Banco de ideas y panel en `index.jsp`

La página verifica estudiante activo y programa asignado; redirige roles distintos a su panel. Sus consultas preparan varias secciones de la pantalla:

- **Catálogo aprobado:** consulta `ideas_aprobadas`, enlaza el director y el evaluador y limita resultados al programa del usuario.
- **Búsqueda:** el parámetro `q` se normaliza, se utiliza como patrón y se busca en título, problema,

descripción, tecnología, programa, área, proponente y director.

- **Disponibilidad:** excluye una selección vigente y las ideas que ya tienen un proyecto final aprobado.
- **Propuestas propias:** lista ideas de tipo estudiantil y obtiene la revisión más reciente mediante `MAX(id)` sobre `revisiones_ideas`.
- **Plazos:** calcula seis meses desde la aprobación de una solicitud para presentar anteproyecto y seis meses desde la aprobación de anteproyecto para el proyecto final.
- **Finalización:** marca el proyecto terminado cuando el proyecto final está aprobado y su revisión final evaluadora fue aprobada.

La vista SQL simplifica el filtro de aprobación; la JSP agrega controles de programa, reserva, asignación y cierre. De esta manera, el catálogo que recibe el estudiante no es simplemente una consulta de todas las ideas registradas.

Este fragmento de `index.jsp` prepara la búsqueda del banco. El programa se toma del perfil cargado y cada término se envía como parámetro a la consulta:

```
SELECT v.id,v.codigo,v.titulo,v.problema,v.descripcion,
  v.recursos_tecnologias,v.programa,v.area,v.proponente,
  director.nombre_completo AS director
FROM ideas_aprobadas v
JOIN ideas i ON i.id=v.id
JOIN direcciones_ideas d ON d.idea_id=i.id
JOIN usuarios director ON director.id=d.director_id
WHERE v.programa_id=?
  AND (i.seleccionada_por IS NULL OR
       (i.seleccionada_hasta IS NOT NULL
        AND i.seleccionada_hasta<=NOW()))
  AND (v.titulo LIKE ? OR v.problema LIKE ?
       OR v.descripcion LIKE ?)
ORDER BY v.aprobada_en DESC,v.codigo
<sql:param value="${activeUser.rows[0].programa_id}" />
<sql:param value="${searchPattern}" />
<sql:param value="${searchPattern}" />
<sql:param value="${searchPattern}" />
```

La consulta completa también compara tecnología, programa, área, proponente y director, y excluye ideas con proyecto final aprobado.

5.2 Paso 11. Solicitar idea del banco

`solicitudes-ideas.jsp` valida la idea seleccionada, el enlace al comprobante y, de haberlo, el código estudiantil del integrante. El enlace debe comenzar por `https://drive.google.com/` o `https://docs.google.com/`, y no superar 500 caracteres. El compañero debe ser estudiante activo del mismo programa.

La acción `request` agrupa en una transacción bloqueo de filas, validación de proceso activo, bloqueo temporal de la idea e inserción/reutilización de la solicitud:

```
UPDATE ideas
```

```
SET seleccionada_por=?, seleccionada_hasta=NULL
WHERE id=? AND estado='aprobada'
  AND programa_id=(SELECT programa_id
                   FROM estudiantes WHERE usuario_id=?)
  AND EXISTS (
    SELECT 1 FROM direcciones_ideas d
    WHERE d.idea_id=ideas.id
  )
  AND (seleccionada_por IS NULL
       OR (seleccionada_hasta IS NOT NULL
           AND seleccionada_hasta<=NOW()))
```

Antes del cambio, la JSP usa `SELECT ... FOR UPDATE` sobre estudiantes y comprueba que el líder y el compañero no tengan otro proceso activo. Tras reservar, si había una solicitud retirada o ya vencida puede reactivarla; si no, inserta una fila en `solicitudes_ideas` con comprobante y estado pendiente.

5.2.1. Reserva y política de tiempo

La reserva queda reflejada en `ideas.seleccionada_por`. La fecha de expiración es `seleccionada_hasta`, pero el código de solicitud la inicializa como nula, lo que mantiene la idea reservada durante la revisión. La coordinación establece una fecha de corrección de tres días cuando rechaza un comprobante; al terminar ese plazo, la selección puede liberarse.

En la acción de solicitud revisada no se encontró la asignación de una expiración inicial de 48 horas. La existencia de la columna `seleccionada_hasta` por sí sola no prueba que se implemente el límite de 48 horas del documento de requisitos.

5.3 Paso 12. Corregir y cancelar una solicitud

- **`correct`:** bloquea al estudiante, y solo actualiza una solicitud suya rechazada cuya fecha de corrección no venció y cuya idea sigue reservada a su nombre. Reemplaza comprobante/mensaje, vuelve el estado a pendiente y pone el vencimiento de selección en nulo mientras se revisa de nuevo.
- **`cancel`:** exige confirmación expresa; valida estudiante, estado y propiedad de la reserva; marca la solicitud retirada y libera la idea dentro de la transacción.

La unicidad de idea/estudiante permite conservar una fila histórica y actualizarla si el usuario vuelve a presentar la solicitud.

En `cancel`, el código primero cambia el estado de la solicitud y solo si se afectó una fila libera la reserva de esa misma idea:

```
UPDATE solicitudes_ideas
SET estado='retirada',corregir_hasta=NULL
WHERE id=? AND estudiante_id=?
  AND estado IN ('pendiente','aprobada','rechazada');

UPDATE ideas
SET seleccionada_por=NULL,seleccionada_hasta=NULL
WHERE id=? AND seleccionada_por=?;
```

Ambas escrituras están dentro de la misma transacción y la liberación se ejecuta de manera condicional al resultado de la primera.

Para la acción `correct`, el cambio queda limitado al estudiante propietario y al plazo abierto:

```
UPDATE solicitudes_ideas s
JOIN ideas i ON i.id=s.idea_id
SET s.comprobante_pago_url=?,s.mensaje=NULLIF(?, ''),
    s.estado='pendiente',s.pago_verificado=0,
    s.corregir_hasta=NULL
WHERE s.id=? AND s.estudiante_id=?
  AND s.estado='rechazada' AND s.corregir_hasta>NOW()
  AND i.seleccionada_por=s.estudiante_id
  AND i.seleccionada_hasta=s.corregir_hasta;
```

5.4 Paso 13. Registrar idea estudiantil en `crear-idea.jsp`

La propuesta del estudiante es una fila de `ideas` con tipo `estudiantil`, no una fila en tabla independiente de ideas externas. Se comprueba título (máximo 120 caracteres), problema (1.200), descripción (1.800), recursos (800), área activa, programa del estudiante y URL compartida del comprobante.

En el POST, la transacción bloquea los registros del líder y del compañero, evita duplicar procesos activos y realiza dos escrituras relacionadas: crea la idea con estado `pendiente_revision` y luego crea la solicitud con URL de comprobante. Un aviso de error cubre fallos al crear cualquiera de los dos registros; el flujo exitoso redirige con mensaje temporal.

La inserción principal obtiene el programa del estudiante desde su perfil y acepta únicamente un área activa:

```
INSERT INTO ideas
  (codigo,titulo,problema,descripcion,recursos_tecnologias,
   tipo,estado,programa_id,area_id,proponente_id,
   seleccionada_por,enviada_en)
SELECT CONCAT('IDEA-',UPPER(SUBSTRING(
         REPLACE(UUID(),'-',''),1,16))),
       ?,?,?,NULLIF(?,''),'estudiantil','pendiente_revision',
       e.programa_id,?,?,?,NOW()
FROM estudiantes e
WHERE e.usuario_id=? AND e.programa_id=?
  AND EXISTS (
    SELECT 1 FROM areas_tematicas a
    WHERE a.id=? AND a.activa=1
  );
```

Cuando esta operación crea una fila, la acción obtiene el identificador con `LAST_INSERT_ID()` e inserta la solicitud y el comprobante en la misma transacción.

6. Recorrido por el código: coordinación y revisión académica

6.1 Paso 14. Coordinación revisa comprobantes y solicitudes

`coordinador-ideas.jsp` admite al coordinador activo. En la acción `reviewIdeaRequest`:

1. Valida que la decisión sea aprobar/rechazar y que las observaciones no superen 1.200 caracteres. Al rechazar exige motivo.
2. Carga la solicitud y la idea unidas y bloquea filas relevantes mediante `FOR UPDATE`, reduciendo cambios concurrentes de la misma solicitud.
3. Para aprobar, confirma que exista evaluador docente activo y que su ID sea diferente al director.
4. Una vez asignado evaluador, actualiza el estado/pago de la solicitud y graba la respuesta, el usuario coordinador y la fecha.
5. Si se rechaza el comprobante, calcula `corregir_hasta` con un intervalo de tres días y sincroniza la fecha de selección de la idea.

El comprobante se revisa como URL asociada a la solicitud; la tabla contiene un indicador `pago_verificado`. El SQL no crea una transacción bancaria ni registra una tarifa monetaria como columna de pago.

La decisión del coordinador actualiza el estado y, si rechaza el comprobante, abre un plazo de tres días. La página también exige un docente evaluador válido y distinto del director al aprobar:

```
UPDATE solicitudes_ideas
SET estado=?,
    pago_verificado=CASE WHEN ?='aprobada' THEN 1 ELSE 0 END,
    respuesta=NULLIF(?,''),
    respondida_por=?,respondida_en=NOW(),
    corregir_hasta=CASE WHEN ?='rechazada'
      THEN DATE_ADD(NOW(),INTERVAL 3 DAY) ELSE NULL END
WHERE id=? AND estado='pendiente';

UPDATE ideas i
JOIN solicitudes_ideas s ON s.idea_id=i.id
SET i.seleccionada_hasta=s.corregir_hasta
WHERE s.id=? AND i.seleccionada_por=s.estudiante_id;
```

En el caso aprobado, otra consulta de la misma transacción registra al evaluador antes de marcar el comprobante verificado:

```
INSERT INTO evaluadores_ideas
  (idea_id,evaluador_id,asignado_por)
SELECT i.id,d.usuario_id,?
FROM ideas i
JOIN direcciones_ideas director ON director.idea_id=i.id
JOIN docentes d ON d.usuario_id=?
JOIN usuarios u ON u.id=d.usuario_id AND u.estado='activo'
JOIN usuario_roles ur ON ur.usuario_id=u.id
JOIN roles r ON r.id=ur.rol_id AND r.nombre='docente'
JOIN solicitudes_ideas s ON s.idea_id=i.id
WHERE s.id=? AND s.estado='pendiente'
  AND i.seleccionada_por=s.estudiante_id
```

```
  AND director.director_id<>d.usuario_id
ON DUPLICATE KEY UPDATE
  evaluador_id=VALUES(evaluador_id),
  asignado_por=VALUES(asignado_por);
```

6.2 Paso 15. Coordinación revisa ideas

La revisión académica permite aprobar o rechazar propuestas y persiste cada decisión en `revisiones_ideas`, con revisor, observaciones y fecha. Para la propuesta estudiantil la página comprueba además que se haya verificado el comprobante, y exige seleccionar un director y un evaluador que no sean la misma persona. El banco público consume únicamente ideas aprobadas mediante la vista `ideas_aprobadas`.

En la rama de aprobación, la propuesta debe seguir pendiente de revisión. Para una idea estudiantil el código además exige una solicitud asociada con comprobante verificado; la decisión queda en la fila de la idea y en su historial:

```
UPDATE ideas
SET estado='aprobada',revisada_en=NOW(),aprobada_en=NOW()
WHERE id=? AND estado IN ('pendiente_revision','en_revision')
  AND (tipo<>'estudiantil' OR EXISTS (
    SELECT 1 FROM solicitudes_ideas s
    WHERE s.idea_id=ideas.id
      AND s.estudiante_id=ideas.proponente_id
      AND s.estado IN ('pendiente','retirada')
      AND s.pago_verificado=1
  ));

INSERT INTO revisiones_ideas
  (idea_id,revisor_id,decision,observaciones)
VALUES (?,?,?,NULLIF(?, ''));
```

En las propuestas estudiantiles, la misma acción registra además director y evaluador docente distintos:

```
INSERT INTO direcciones_ideas
  (idea_id,director_id,asignado_por)
VALUES (?,?,?)
ON DUPLICATE KEY UPDATE
  director_id=VALUES(director_id),
  asignado_por=VALUES(asignado_por);

INSERT INTO evaluadores_ideas
  (idea_id,evaluador_id,asignado_por)
SELECT i.id,d.usuario_id,?
FROM ideas i
JOIN docentes d ON d.usuario_id=?
JOIN usuarios u ON u.id=d.usuario_id AND u.estado='activo'
JOIN usuario_roles ur ON ur.usuario_id=u.id
JOIN roles r ON r.id=ur.rol_id AND r.nombre='docente'
JOIN direcciones_ideas director ON director.idea_id=i.id
WHERE i.id=? AND director.director_id<>d.usuario_id;
```

En caso de rechazo, actualiza el estado y libera la selección estudiantil dentro de una transacción.

6.3 Paso 16. Docentes proponen ideas institucionales

La acción `proposeIdea` de `docente.jsp` comprueba el formulario y que programa y área sigan activos. Dentro de una transacción:

1. Inserta la idea como `institucional` y `pendiente_revision`, genera un código a partir de UUID y enlaza programa, área y proponente.
2. Obtiene el ID creado con `LAST_INSERT_ID()`.
3. Inserta la relación de director en `direcciones_ideas`, si el perfil docente corresponde al usuario autenticado.

Así, el registro de la propuesta y la asignación inicial de dirección quedan en la misma transacción.

```
<sql:transaction dataSource="jdbc/ProjectsDB">
  <sql:update var="createdIdea">
    INSERT INTO ideas
      (codigo,titulo,problema,descripcion,recursos_tecnologias,
       tipo,estado,programa_id,area_id,proponente_id,enviada_en)
    VALUES
      (CONCAT('IDEA-',UPPER(SUBSTRING(
        REPLACE(UUID(),'-',''),1,16))),?,?,?,NULLIF(?, ''),
       'institucional','pendiente_revision',?,?,?,NOW())
    <sql:param value="${fn:trim(param.title)}" />
    <sql:param value="${fn:trim(param.problem)}" />
    <sql:param value="${fn:trim(param.description)}" />
    <sql:param value="${fn:trim(param.resources)}" />
    <sql:param value="${param.programId}" />
    <sql:param value="${param.areaId}" />
    <sql:param value="${sessionScope.userId}" />
  </sql:update>
  <sql:query var="createdIdeaId">
    SELECT LAST_INSERT_ID() AS id
  </sql:query>
  <sql:update>
    INSERT INTO direcciones_ideas
      (idea_id,director_id,asignado_por)
    SELECT ?,d.usuario_id,d.usuario_id
    FROM docentes d WHERE d.usuario_id=?
    <sql:param value="${createdIdeaId.rows[0].id}" />
    <sql:param value="${sessionScope.userId}" />
  </sql:update>
</sql:transaction>
```

Los parámetros del formulario se pasan a las etiquetas SQL como valores separados; no se concatenan a la sentencia.

6.4 Paso 17. Revisión del anteproyecto

Los docentes autorizados procesan `reviewAnteproyecto`. Una consulta de bloqueo permite revisar solo un anteproyecto pendiente de director asignado o pendiente de evaluador asignado. La primera aprobación cambia el estado a `pendiente_evaluador`; la decisión evaluadora puede aprobar, aplazar o rechazar. Después de cambiar el estado se inserta una fila en `revisiones_anteproyectos` con etapa, observación y fecha.

La transición se aplica solo si el identificador y la asignación de rol coinciden. Si aprueba el director, la etapa pasa al evaluador; todas las demás decisiones quedan como estado final de esa revisión:

```
UPDATE anteproyectos
SET estado=CASE
  WHEN estado='pendiente_revision' AND ?='aprobado'
    THEN 'pendiente_evaluador'
  ELSE ?
END,revisado_en=NOW()
WHERE id=? AND (
  (director_id=? AND estado='pendiente_revision')
  OR (estado='pendiente_evaluador' AND EXISTS (
    SELECT 1 FROM evaluadores_ideas ei
    WHERE ei.idea_id=anteproyectos.idea_id
      AND ei.evaluador_id=?
  ))
);

INSERT INTO revisiones_anteproyectos
  (anteproyecto_id,director_id,etapa,decision,observaciones)
VALUES (?,?,?,?,NULLIF(?, ''));
```

La JSP inserta la revisión únicamente cuando el `UPDATE` afectó una fila y envuelve ambos cambios en la transacción.

6.5 Paso 18. Revisión del proyecto final

`reviewFinalProject` realiza comprobación equivalente: relaciona el documento con su anteproyecto/idea y comprueba que quien actúa sea director en etapa inicial o evaluador en la segunda. El cambio autorizado actualiza `proyectos_finales.estado`, y la decisión se agrega a `revisiones_proyectos_finales`. Cuando el director aprueba el trabajo se espera la evaluación; el resultado final aprobado por el evaluador cierra el ciclo del alumno en el panel.

```
UPDATE proyectos_finales pf
JOIN anteproyectos ap ON ap.id=pf.anteproyecto_id
LEFT JOIN direcciones_ideas d ON d.idea_id=ap.idea_id
LEFT JOIN evaluadores_ideas ei ON ei.idea_id=ap.idea_id
SET pf.estado=CASE
  WHEN pf.estado='pendiente_director' AND ?='aprobado'
    THEN 'pendiente_evaluador'
  ELSE ?
END,pf.revisado_en=NOW()
WHERE pf.id=? AND (
```

```
  (pf.estado='pendiente_director' AND d.director_id=?)
  OR (pf.estado='pendiente_evaluador' AND ei.evaluador_id=?)
);

INSERT INTO revisiones_proyectos_finales
  (proyecto_final_id,revisor_id,etapa,decision,observaciones)
VALUES (?,?,?,?,NULLIF(?, ''));
```

7. Entregas estudiantiles y seguimiento

7.1 Paso 19. Enviar anteproyecto y proyecto final

`anteproyecto.jsp` protege la página con sesión y perfil estudiante. El formulario requiere título, descripción y URL de Drive/Docs (hasta 500 caracteres). Las operaciones usan un `INSERT ... SELECT` con restricciones a la vez que los datos llegan del formulario:

- El anteproyecto solo se crea si hay una solicitud aprobada, la idea está aprobada y seleccionada por ese estudiante, el programa coincide y la idea tiene director.
- El proyecto final solo se crea si el anteproyecto pertenece al usuario y está aprobado y la solicitud de la idea está aprobada.
- La corrección de anteproyecto y de proyecto final se limita a filas propias aplazadas.

La condición de propiedad/estado vive en el SQL, además de la comprobación de sesión de la página.

El envío del proyecto final está implementado como una inserción condicionada: exige un anteproyecto aprobado del estudiante y mantiene enlazada la solicitud aprobada y la idea seleccionada.

```
INSERT INTO proyectos_finales
  (anteproyecto_id,titulo,descripcion,enlace_drive)
SELECT ap.id,?,?,?
FROM anteproyectos ap
JOIN solicitudes_ideas s
  ON s.idea_id=ap.idea_id
 AND s.estudiante_id=ap.estudiante_id
 AND s.estado='aprobada'
JOIN ideas i ON i.id=ap.idea_id
  AND i.estado='aprobada'
  AND i.seleccionada_por=ap.estudiante_id
JOIN usuarios u ON u.id=ap.estudiante_id
  AND u.estado='activo'
WHERE ap.id=? AND ap.estudiante_id=?
  AND ap.estado='aprobado';
```

7.2 Paso 20. Revisar y reenviar una entrega aplazada

En una corrección, la página renueva título, descripción y enlace; reinicia el estado a `pendiente_revision` o `pendiente_director`, según fase, y restablece fecha de envío/revisión. El registro de la

revisión anterior permanece en su tabla de historial, lo que permite consultar la decisión previa.

Para reenviar un anteproyecto aplazado, la actualización queda restringida a la fila propia y solo cambia una entrega cuyo estado sea `aplazado`:

```
UPDATE anteproyectos
SET titulo=?,descripcion=?,enlace_drive=?,
    estado='pendiente_revision',
    enviado_en=NOW(),revisado_en=NULL
WHERE id=? AND estudiante_id=? AND estado='aplazado';
```

El proyecto final utiliza una rama equivalente y lo devuelve a `pendiente_director` para reiniciar la revisión según la fase prevista por la JSP.

7.3 Fases visibles en `project-phase-tracker.jsp`

El fragmento incluido desde los paneles de seguimiento recibe la fase actual y dibuja seis pasos con clases distintas para etapas completadas, actual y pendientes. Ese componente solo construye la representación visual; la consulta de procesos y estados proviene de `docente.jsp`, `anteproyecto.jsp` o `index.jsp`.

8. Código del proyecto: datos y operaciones principales

8.1 Paso 21. Cargar las ideas semilla desde SQL

`database/02_insert_initial_data.sql` define diez ideas de ejemplo y luego las inserta únicamente cuando encuentra docentes, roles, programas y áreas activos. El fragmento muestra esa carga condicional: evita duplicar códigos ya presentes y conecta cada propuesta con su docente y catálogos.

```
INSERT INTO ideas (
  codigo,titulo,problema,descripcion,objetivos,
  recursos_tecnologias,beneficiarios,tipo,estado,
  programa_id,area_id,proponente_id,enviada_en
)
SELECT seed.codigo,seed.titulo,seed.problema,seed.descripcion,
  seed.objetivos,seed.recursos_tecnologias,seed.beneficiarios,
  'institucional','pendiente_revision',seed.programa_id,
  seed.area_id,seed.proponente_id,NOW()
FROM seed_teacher_ideas seed
JOIN usuarios u ON u.id=seed.proponente_id AND u.estado='activo'
JOIN docentes d ON d.usuario_id=u.id
JOIN usuario_roles ur ON ur.usuario_id=u.id
JOIN roles r ON r.id=ur.rol_id AND r.nombre='docente'
JOIN programas_academicos p
  ON p.id=seed.programa_id AND p.activo=1
JOIN areas_tematicas a ON a.id=seed.area_id AND a.activa=1
WHERE NOT EXISTS (
  SELECT 1 FROM ideas existing WHERE existing.codigo=seed.codigo
);
```

El script no crea las cuentas docentes. Por eso, ejecutar el bloque no garantiza que se inserten las diez

ideas: primero deben existir los usuarios y sus perfiles/roles, además de los catálogos que coincidan con las referencias de la tabla temporal.

8.2 Reserva y solicitud de una idea en `solicitudes-ideas.jsp`

Antes de reservar, la página bloquea los registros de los estudiantes que forman el equipo y verifica que no tengan otro proceso activo. La actualización condicionada comprueba el estado aprobado, el programa del estudiante y la asignación de director; si la reserva tiene éxito, se registra la solicitud con el comprobante.

```
<sql:update var="reservedIdea">
  UPDATE ideas
  SET seleccionada_por=?,seleccionada_hasta=NULL
  WHERE id=? AND estado='aprobada'
    AND programa_id=(
      SELECT programa_id FROM estudiantes WHERE usuario_id=?
    )
    AND EXISTS (
      SELECT 1 FROM direcciones_ideas d WHERE d.idea_id=ideas.id
    )
    AND (seleccionada_por IS NULL OR
      (seleccionada_hasta IS NOT NULL AND seleccionada_hasta<=NOW()))
  <sql:param value="${sessionScope.userId}" />
  <sql:param value="${param.ideaId}" />
  <sql:param value="${sessionScope.userId}" />
</sql:update>

<c:if test="${reservedIdea eq 1}">
  <sql:update var="createdRequest">
    INSERT INTO solicitudes_ideas
      (idea_id,estudiante_id,integrante_id,mensaje,
       comprobante_pago_url,estado)
    VALUES (?,?,?,NULLIF(?,''),?,'pendiente')
    <sql:param value="${param.ideaId}" />
    <sql:param value="${sessionScope.userId}" />
    <sql:param value="${empty param.partnerCode ? null : partnerId}" />
    <sql:param value="${fn:trim(param.message)}" />
    <sql:param value="${fn:trim(param.receiptUrl)}" />
  </sql:update>
</c:if>
```

El bloque ocurre dentro de `<sql:transaction>`. La columna `seleccionada_hasta` se establece en nulo al reservar; el código revisado no implementa aquí un vencimiento inicial de 48 horas.

8.3 Envío de anteproyecto desde `anteproyecto.jsp`

El formulario no inserta una entrega solo por recibir datos. La consulta crea el anteproyecto mediante un `INSERT ... SELECT` que exige solicitud aprobada, idea aprobada del mismo programa, selección del estudiante, director asignado y cuenta activa.

```
INSERT INTO anteproyectos
  (idea_id,estudiante_id,director_id,titulo,descripcion,enlace_drive)
SELECT i.id,?,d.director_id,?,?,?
FROM ideas i
JOIN direcciones_ideas d ON d.idea_id=i.id
JOIN estudiantes e ON e.usuario_id=?
JOIN solicitudes_ideas s
  ON s.idea_id=i.id
 AND s.estudiante_id=e.usuario_id
 AND s.estado='aprobada'
WHERE i.id=?
  AND i.estado='aprobada'
  AND i.programa_id=e.programa_id
  AND i.seleccionada_por=e.usuario_id
  AND EXISTS (
    SELECT 1 FROM usuarios u
    WHERE u.id=e.usuario_id AND u.estado='activo'
  )
```

Si cualquiera de esas condiciones falla, el `INSERT ... SELECT` no crea filas y la página informa que la idea no está disponible para ese estudiante. La JSP valida además el formato Drive/Docs y la longitud del enlace.

8.4 Revisión de anteproyecto y escritura del historial

En `docente.jsp`, la revisión se realiza dentro de una transacción. La primera aprobación del director cambia el estado a `pendiente_evaluador`; solo después de confirmar que se actualizó una fila se inserta el registro de revisión con etapa, decisión y observaciones.

```
UPDATE anteproyectos
SET estado=CASE
  WHEN estado='pendiente_revision' AND ?='aprobado'
    THEN 'pendiente_evaluador'
  ELSE ?
END,revisado_en=NOW()
WHERE id=? AND (
  (director_id=? AND estado='pendiente_revision')
  OR (estado='pendiente_evaluador' AND EXISTS (
    SELECT 1 FROM evaluadores_ideas ei
    WHERE ei.idea_id=anteproyectos.idea_id
      AND ei.evaluador_id=?
  ))
);

INSERT INTO revisiones_anteproyectos
  (anteproyecto_id,director_id,etapa,decision,observaciones)
VALUES (?,?,?,?,NULLIF(?, ''));
```

La página comprueba el rol/identidad asignada antes de aplicar el cambio y conserva cada decisión en `revisiones_anteproyectos`; así, el estado actual de la entrega y el historial de revisiones quedan representados por separado.

9. Puesta en marcha paso a paso

9.1 Paso 22. Instalar y cargar la base

1. Iniciar MariaDB/MySQL compatible con las construcciones del script.
2. Ejecutar `database/01_create_schema.sql`. El usuario de instalación necesita permiso para crear la función almacenada.
3. Preparar cuentas docentes 10 y 25 con estado activo, perfil y rol correctos si se quieren importar todas las ideas de muestra.
4. Ejecutar `database/02_insert_initial_data.sql` y comprobar programas, áreas, ideas insertadas y sus asignaciones.
5. Recordar que el SQL de seed no inserta usuarios ni genera contraseñas de prueba.

9.2 Paso 23. Configurar Tomcat y JNDI

`web.xml` declara `jdbc/ProjectsDB` como `javax.sql.DataSource`. La configuración de contexto debe exponer exactamente el mismo JNDI, seleccionar el controlador MySQL disponible y apuntar a la base recién creada. Configure credenciales en el entorno del servidor; no las incluya en este informe ni las publique en el código fuente.

Compruebe que JSTL 1.2 sea visible para el webapp, y que Tomcat pueda cargar el conector `com.mysql.cj.jdbc.Driver`. Si se utiliza un Tomcat con API `jakarta.servlet`, primero se debe evaluar la migración del descriptor y las dependencias en vez de asumir que el proyecto actual compila sin cambios.

9.3 Paso 24. Guion de prueba por roles

Preparación: ejecutar el guion en una base de pruebas, con Tomcat y JNDI `jdbc/ProjectsDB`; no utilizar cuentas o datos de producción. Crear cuentas de prueba activas para administrador, coordinación, estudiantes y docentes. Asignar perfil y programa a cada estudiante, y perfil/rol a cada docente; para repetir la carga seed, cumplir los requisitos de docentes del paso 21. Mantener dos sesiones independientes para los casos de reserva y permisos.

#	Usuario	Acción	Qué debe ocurrir
1	Administrador	Iniciar sesión con una cuenta activa y abrir Gestión de usuarios.	Se redirige a `admin-usuarios.jsp`; la sesión conserva el rol de administrador.
2	Administrador	Crear una cuenta estudiante con correo, contraseña, código y programa activos.	La cuenta, el rol y el perfil estudiante quedan creados; el usuario aparece en el listado.
3	Administrador	Intentar crear un usuario con correo duplicado y luego desactivar al último administrador.	El duplicado no se registra y la cuenta del último administrador permanece activa; la página muestra el aviso correspondiente.

#	Usuario	Acción	Qué debe ocurrir (continuación)
4	Todos los roles	Cerrar sesión y volver a una página privada con la URL directa.	La sesión se elimina y la página protegida redirige a login; cada rol autorizado llega a su panel al autenticarse.
5	Estudiante A	Abrir el banco, buscar una palabra del título y revisar ideas aprobadas.	Se muestran coincidencias del programa del estudiante; no se muestran ideas de otro programa ni una idea reservada vigente.
6	Estudiante A	Solicitar una idea libre con enlace de comprobante Drive/Docs válido y, opcionalmente, compañero activo del mismo programa.	Se crea una solicitud pendiente y se reserva la idea. El compañero de otro programa o una cuenta inactiva no es aceptado.
7	Estudiante B	En una sesión distinta, intentar solicitar la misma idea que reservó Estudiante A.	La reserva condicionada impide crear la segunda solicitud; la idea continúa asociada a Estudiante A.
8	Estudiante A y compañero	Intentar iniciar otra solicitud mientras alguno ya tiene un proceso pendiente/aprobado o una corrección abierta.	El sistema detecta el proceso activo y no permite registrar una solicitud adicional para ninguno de los integrantes.
9	Estudiante A	Cancelar una solicitud propia y confirmar explícitamente la operación.	La solicitud pasa a `retirada` y la idea queda libre para otra persona. Sin confirmación, no se cancela.
10	Estudiante A	Enviar un comprobante con URL no admitida o de más de 500 caracteres.	La validación rechaza el formulario y no crea ni modifica la solicitud.
11	Coordinador	Rechazar un comprobante pendiente agregando observación.	La solicitud queda rechazada, se fija plazo de corrección de tres días y la reserva queda vinculada a ese plazo.
12	Estudiante A	Reenviar el comprobante durante el plazo de corrección y repetir después de vencido.	Dentro del plazo vuelve a pendiente y el enlace se actualiza; tras el vencimiento, no se admite la corrección.
13	Coordinador	Verificar un comprobante y asignar un evaluador docente diferente del director.	Se marca el pago como verificado, se registra el coordinador y el evaluador; la revisión no permite asignar al director como evaluador.
14	Estudiante A	Crear una propuesta estudiantil con área activa y comprobante válido.	En una transacción se crean la idea estudiantil y su solicitud; ambas quedan relacionadas con el estudiante.
15	Coordinador	Revisar una propuesta estudiantil con comprobante verificado, seleccionar director y evaluador distintos y aprobarla.	La idea cambia a aprobada, se asignan los docentes y se agrega el evento a `revisiones_ideas`; aparece en el banco correspondiente.
16	Docente	Registrar una propuesta institucional con programa y área activos.	Se crea como `pendiente_revision`, se asigna el docente proponente como director cuando tiene perfil docente y la coordinación puede revisarla.
17	Estudiante A	Enviar anteproyecto para una idea aprobada, seleccionada y con solicitud aprobada.	El anteproyecto queda creado como pendiente de revisión. Para una idea no propia, no aprobada o sin director, no se inserta.
18	Director y evaluador	Aprobar el anteproyecto primero como director y luego revisarlo como evaluador.	La aprobación del director mueve el estado a `pendiente_evaluador`; cada decisión autorizada queda registrada en `revisiones_anteproyectos`.

#	Usuario	Acción	Qué debe ocurrir (continuación)
19	Docente no asignado	Intentar revisar el anteproyecto o proyecto final mediante una URL/formulario directo.	La operación no modifica el registro porque el usuario no corresponde al director/evaluador asignado.
20	Estudiante A	Enviar un proyecto final después de la aprobación del anteproyecto; probar también un anteproyecto ajeno o no aprobado.	El caso válido crea el proyecto final; el caso no autorizado no inserta filas.
21	Director y evaluador	Revisar el proyecto final en las dos etapas, usando aprobado, aplazado y rechazado en registros de prueba independientes.	El flujo avanza del director al evaluador cuando se aprueba; cada decisión se registra en `revisiones_proyectos_finales`.
22	Estudiante A	Corregir y reenviar una entrega aplazada; después intentar modificar una entrega aprobada o de otro estudiante.	Solo la entrega propia aplazada vuelve a revisión; el historial anterior permanece. Los otros cambios no afectan filas.

Este es un guion ejecutable propuesto a partir de las acciones y validaciones encontradas en el código. Los resultados de la tabla son esperados; no se presentan como pruebas funcionales ya realizadas. El informe fue revisado estáticamente y no se ejecutó el sistema con Tomcat y una base de datos activa.

10. Errores frecuentes y diagnóstico

Síntoma	Qué revisar
No se encuentra `jdbc/ProjectsDB`	Comparar el nombre en `web.xml`, configuración Tomcat y cada etiqueta `dataSource`.
Falla la conexión SQL	Revisar base activa, usuario/permisos, controlador MySQL disponible, URL de conexión y configuración externa de credenciales.
Error en etiquetas JSTL	Confirmar `jstl-1.2.jar` en `WEB-INF/lib` y el descriptor compatible con la versión del contenedor.
No se insertan las diez ideas	Validar IDs 10 y 25: usuario activo, perfil docente, rol docente; comprobar además que programa y área estén activos.
Falla al crear un usuario	Consultar duplicidad de correo/código y validar rol, longitud de contraseña, programa y perfil exigido.
La idea no aparece en el banco	Revisar estado aprobado, programa del estudiante, director asignado, reserva activa y si ya existe un proyecto final aprobado.
Solicitud rechazada por compañero	Verificar que el código corresponda a cuenta activa de estudiante del mismo programa y que el equipo no tenga otro proceso activo.
Enlace de documento no aceptado	Comprobar prefijo Drive/Docs compartible y límite de caracteres esperado por esa pantalla. El flujo pide URL; no archivo binario.
JSP no compila en Tomcat reciente	Revisar la diferencia de API `javax.servlet/jakarta.servlet`; esta entrega no demuestra migración a Tomcat 10/11.

11. Requisitos frente a la implementación

Tema del documento de requisitos	Evidencia revisada	Conclusión acotada al código entregado
Acceso	Login, cuatro roles seed y guardas en cada JSP privada	Se observan rutas para estudiante, docente, coordinación y administración; director/evaluador son asignaciones de docentes.
Usuarios	Crear, editar y desactivar en `admin-usuarios.jsp`	Incluye validaciones, transacciones, hash con sal y resguardo del último administrador.
Banco de ideas	Vista aprobada, propuestas y revisiones de los roles	El flujo usa `index.jsp`, `crear-idea.jsp`, `docente.jsp` y `coordinador-ideas.jsp`.
Créditos	No se encontró campo de créditos en `estudiantes` ni regla equivalente	No se puede confirmar la validación del requisito en este código.
Reserva	La selección se crea con límite nulo; la corrección usa tres días	No se observa asignación inicial explícita de 48 horas.
Pago	URL de comprobante y bandera `pago_verificado`	No se observan monto, servicio de pago ni tabla específica de pagos.
Adjuntos	El ER indica el límite de 15 MB	Las páginas aceptan enlaces web; no se encontró carga binaria ni validación de tamaño.
Entregas	Formularios, estados, revisores y correcciones	El código valida propiedad/estado en SQL y registra decisiones por etapa.
Seguimiento	Tablas de proyectos, asesores, integrantes, entregables, avisos e historial	Están declaradas en esquema; verificar los paneles antes de afirmar un CRUD completo.

Las conclusiones de esta tabla describen los archivos de la carpeta inspeccionada. No deben generalizarse a otras ramas, instalaciones o entregas no incluidas en el proyecto disponible.

12. Conclusiones

El código entregado implementa un recorrido reconocible desde el usuario y el banco de ideas hasta la verificación de solicitud, el anteproyecto y la evaluación final. Sus detalles importantes están en las comprobaciones del lado servidor JSP, los estados guardados en SQL, las claves foráneas y la agrupación de escrituras relacionadas en transacciones.

En esta nueva versión se incluye el modelo entidad-relación original de los requerimientos y se explica por separado el modelo físico que realmente crea el SQL. Esa comparación evita confundir entidades conceptuales de pago, selección o documento con tablas que no existen en la implementación actual.

El desarrollo del informe sigue el enfoque paso a paso del ejemplo de referencia: inventario del proyecto, preparación de base y conexión, recorrido de cada rol/página, fragmentos reales de SQL, datos de

muestra, despliegue y pruebas. La compilación de este documento no sustituye pruebas funcionales: antes de demostrar el sistema se debe validar con Tomcat y una base inicializada, especialmente los prerequisitos del seed y los requerimientos de créditos, reserva y manejo de archivos.

Fuentes e inventario consultados

- Acceso y consulta: `login.jsp`, `logout.jsp` e `index.jsp`.
- Propuestas y solicitudes: `crear-idea.jsp`, `solicitudes-ideas.jsp` y `coordinador-ideas.jsp`.
- Roles y entregas: `docente.jsp`, `anteproyecto.jsp`, `admin-usuarios.jsp` y `usuarios-publicos.jsp`.
- Base de datos: `database/01_create_schema.sql`, `database/02_insert_initial_data.sql` y migraciones de `database/legacy/`.
- Configuración y recursos: `WEB-INF/web.xml`, `WEB-INF/project-phase-tracker.jsp`, `META-INF/context.xml`, `WEB-INF/lib/` y `assets/css/styles.css`.
- `REQUERIMIENTOS.pdf`: requerimientos funcionales/no funcionales y figura del modelo entidad-relación reproducida en este informe.
- `Informe_Parcial.pdf`: referencia de estructura explicativa paso a paso y presentación.